

# Predykcja nowych lokalizacji Żabki w Polsce

Autorzy: Oskar Wiktorowicz i Jakub Truszczyński

## Predykcja nowych lokalizacji Żabek

Miasto

Warszawa

Wynik

94.0%

Pokazane punkty

3

Mieszkańcy

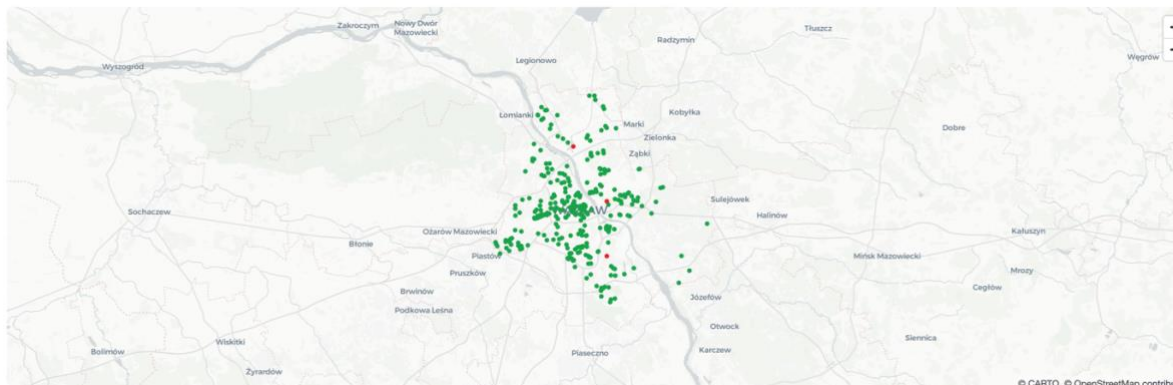
1 864 035

Żabki / 10 tys.

1.62

Wynik jest scorem modelu dla atrakcyjności lokalizacji, a nie gwarancją realnego otwarcia sklepu. Model bazuje na obecnych lokalizacjach i cechach przestrzennych.

● Istniejące Żabki ● Przewidywane lokalizacje



# Spis treści

|                                              |           |
|----------------------------------------------|-----------|
| <b>1. Opis problemu .....</b>                | <b>3</b>  |
| <b>2. Opis danych .....</b>                  | <b>3</b>  |
| <b>3. Przygotowanie danych.....</b>          | <b>5</b>  |
| 3.1 Czyszczenie danych .....                 | 5         |
| 3.2 Tworzenie zbioru treningowego .....      | 5         |
| 3.3 Inżynieria cech .....                    | 6         |
| 3.4 Struktura projektu.....                  | 7         |
| 3.5 Architektura systemu .....               | 8         |
| <b>4. Sposób uruchomienia projektu .....</b> | <b>9</b>  |
| 4.1 Instalacja lokalna .....                 | 9         |
| 4.2 Uruchomienie pipeline Kedro .....        | 9         |
| 4.3 Uruchomienie API i mapy .....            | 9         |
| 4.4 Uruchomienie interfejsu Streamlit .....  | 10        |
| 4.5 Uruchomienie projektu przez Docker ..... | 10        |
| <b>5. Model uczenia maszynowego .....</b>    | <b>11</b> |
| 5.1 Model bazowy .....                       | 11        |
| 5.2 Model docelowy .....                     | 11        |
| 5.3 Interpretacja wyniku modelu .....        | 12        |
| 5.4 Porównanie modeli AutoGluon.....         | 12        |
| <b>6. API i monitoring.....</b>              | <b>14</b> |
| <b>7. Podsumowanie .....</b>                 | <b>15</b> |

# 1. Opis problemu

Projekt dotyczy analizy lokalizacji sklepów sieci Żabka w Polsce oraz przygotowania rozwiązania wspierającego ocenę potencjalnych miejsc pod nowe sklepy. W praktyce wybór dobrej lokalizacji nie zależy tylko od samego adresu, ale od wielu czynników związanych z otoczeniem danego punktu.

Na atrakcyjność lokalizacji mogą wpływać między innymi:

- położenie sklepu na mapie,
- bliskość centrum miasta,
- liczba mieszkańców w danej miejscowości,
- zagęszczenie istniejących sklepów Żabka,
- dostępność punktu dla potencjalnych klientów,
- charakter okolicy, w której znajduje się lokalizacja.

Celem projektu było stworzenie aplikacji wykorzystującej uczenie maszynowe do oceny, czy dana lokalizacja może być odpowiednia dla sklepu Żabka. Projekt obejmuje nie tylko sam model, ale również przygotowanie danych, przetwarzanie ich w pipeline, udostępnienie wyników przez API oraz prosty interfejs użytkownika.

# 2. Opis danych

W projekcie wykorzystano własny dataset przygotowany na potrzeby zadania. Dane o lokalizacjach sklepów Żabka zostały pobrane z wykorzystaniem Overpass API, czyli narzędzia pozwalającego pobierać dane z OpenStreetMap. Wyniki zapisano w formacie GeoJSON, ponieważ dobrze nadaje się on do przechowywania danych przestrzennych, takich jak punkty na mapie oraz ich współrzędne geograficzne.

W projekcie wykorzystano głównie następujące pliki:

- `zabka_locs.geojson` — główny zbiór danych zawierający lokalizacje sklepów Żabka pobrane z OpenStreetMap,
- `city_population.csv` — plik pomocniczy z informacjami o liczbie mieszkańców miast,
- `extra_city_centers.csv` — dodatkowy plik z informacjami o centrach wybranych miast,
- dane pośrednie generowane przez pipeline Kedro — wykorzystywane podczas przygotowania danych dla modelu.

Głównym problemem przy pracy z danymi było to, że dataset pochodził z rzeczywistego źródła, dlatego nie był od razu idealnie dopasowany do projektu. Dane z OpenStreetMap mogą mieć różną strukturę, niepełne atrybuty albo wymagać dodatkowego przekształcenia. Z tego powodu program musiał zostać dopasowany do formatu własnego datasetu.

W ramach przygotowania danych wykonano między innymi:

- wczytanie danych z pliku GeoJSON,
- odczytanie współrzędnych geograficznych sklepów,
- przekształcenie danych przestrzennych do formy tabelarycznej,
- połączenie danych o lokalizacjach z dodatkowymi informacjami o miastach,
- przygotowanie cech wejściowych dla modelu uczenia maszynowego,
- zapisanie danych wynikowych w kolejnych katalogach projektu Kedro.

Dzięki temu własny dataset z Overpass API mógł zostać wykorzystany w dalszych etapach projektu, takich jak trenowanie modelu, tworzenie rankingu kandydackich lokalizacji oraz prezentacja wyników na mapie.

Ograniczeniem datasetu jest to, że dane pochodzą z OpenStreetMap, więc ich jakość zależy od kompletności i aktualności danych społecznościowych. Dodatkowo przykłady negatywne zostały wygenerowane sztucznie, dlatego wynik modelu należy traktować jako ocenę podobieństwa lokalizacji, a nie jednoznaczną decyzję biznesową.

Charakterystyka danych w projekcie w skrócie:

| Parametr                              | Wartość                                        |
|---------------------------------------|------------------------------------------------|
| Źródło danych                         | Overpass API / OpenStreetMap                   |
| Format głównego zbioru                | GeoJSON                                        |
| Główny plik danych                    | zabka_locs.geojson                             |
| Dane pomocnicze                       | city_population.csv,<br>extra_city_centers.csv |
| Liczba rekordów w profilu treningowym | 23 514                                         |
| Liczba cech wejściowych               | 19                                             |
| Typ problemu                          | Klasyfikacja binarna                           |
| Przykłady pozytywne                   | Istniejące lokalizacje sklepów Żabka           |
| Przykłady negatywne                   | Wygenerowane punkty kandydackie                |
| Główna metryka modelu                 | ROC AUC                                        |

## 3. Przygotowanie danych

Przygotowanie danych było jednym z ważniejszych etapów projektu, ponieważ wykorzystany dataset pochodził z rzeczywistego źródła, czyli OpenStreetMap/Overpass. Dane nie były więc od razu gotowe do użycia w modelu i wymagały przekształcenia do formy tabelarycznej.

### 3.1 Czyszczenie danych

Pierwszym etapem było wczytanie pliku `zabka_locs.geojson` oraz wybranie tylko tych obiektów, które posiadały poprawną geometrię typu punkt. Dzięki temu w dalszych etapach wykorzystywane były wyłącznie lokalizacje sklepów możliwe do umieszczenia na mapie.

W ramach czyszczenia danych wykonano między innymi:

- odrzucenie obiektów bez współrzędnych geograficznych,
- odrzucenie geometrii innych niż punkt,
- pobranie długości i szerokości geograficznej,
- ujednolicenie podstawowych pól, takich jak miasto, ulica i kod pocztowy,
- usunięcie duplikatów na podstawie współrzędnych.

Dzięki temu dane z formatu GeoJSON zostały przekształcone do prostszej tabeli, która mogła zostać wykorzystana przez pipeline Kedro.

### 3.2 Tworzenie zbioru treningowego

Ponieważ projekt dotyczył oceny atrakcyjności nowych lokalizacji, należało przygotować zbiór treningowy w sposób sztuczny na podstawie obecnej sieci sklepów. Istniejące lokalizacje Żabek zostały potraktowane jako przykłady pozytywne, czyli miejsca, w których sklep już funkcjonuje.

Następnie wygenerowano siatkę punktów kandydackich na obszarze Polski. Punkty te reprezentowały potencjalne miejsca, w których można byłoby rozważać nową lokalizację. Część punktów oddalonych od istniejących sklepów została wykorzystana jako przykłady negatywne.

W ten sposób powstał problem klasyfikacji binarnej:

| Klasa | Znaczenie                                         |
|-------|---------------------------------------------------|
| 1     | lokalizacja podobna do istniejących sklepów Żabka |
| 0     | lokalizacja kandydacka / negatywna                |

### 3.3 Inżynieria cech

Na podstawie lokalizacji sklepów oraz danych pomocniczych przygotowano cechy opisujące każdy punkt. Cechy te miały pomóc modelowi ocenić, czy dana lokalizacja jest podobna do miejsc, w których obecnie znajdują się sklepy Żabka.

Przykładowe cechy użyte w projekcie:

| Cecha                                           | Opis                                                |
|-------------------------------------------------|-----------------------------------------------------|
| <code>nearest_store_km</code>                   | odległość do najbliższego istniejącego sklepu Żabka |
| <code>stores_1km</code>                         | liczba sklepów Żabka w promieniu 1 km               |
| <code>stores_3km</code>                         | liczba sklepów Żabka w promieniu 3 km               |
| <code>stores_5km</code>                         | liczba sklepów Żabka w promieniu 5 km               |
| <code>nearest_city_center_km</code>             | odległość do najbliższego centrum miasta            |
| <code>city_population</code>                    | liczba mieszkańców najbliższego miasta              |
| <code>stores_per_10k_residents</code>           | liczba sklepów na 10 tysięcy mieszkańców            |
| <code>residents_per_store</code>                | liczba mieszkańców przypadająca na jeden sklep      |
| <code>lon_sin, lon_cos, lat_sin, lat_cos</code> | zakodowane współrzędne geograficzne                 |

Tak przygotowane dane zostały zapisane w katalogach projektu Kedro i wykorzystane podczas trenowania modelu uczenia maszynowego.

### 3.4 Struktura projektu

|   |                          |
|---|--------------------------|
| ▼ | conf                     |
| ▼ | base                     |
|   | ! catalog.yml            |
|   | ! parameters.yml         |
| > | local                    |
| ▼ | data                     |
| ▼ | 01_raw                   |
|   | x city_population.csv    |
|   | x extra_city_centers.csv |
| ▼ | 02_intermediate          |
| ▼ | 05_model_input           |
| ▼ | 06_models                |
| ▼ | 07_model_output          |
| ▼ | 08_reporting             |
|   | ! docker-compose.yml     |
|   | Dockerfile               |
| ▼ | notebooks                |
|   | 01_baseline.ipynb        |
|   | pyproject.toml           |
|   | ↓ README.md              |
|   | requirements.txt         |
| ▼ | src                      |
| ▼ | zabki_prediction         |
|   | __init__.py              |
| > | api                      |
|   | pipeline_registry.py     |
| ▼ | pipelines                |
|   | __init__.py              |
| > | site_selection           |
|   | settings.py              |
| ▼ | ui                       |
|   | __init__.py              |
|   | streamlit_app.py         |
| ▼ | zabki_w_polsce.egg-info  |
|   | dependency_links.txt     |
|   | entry_points.txt         |
|   | PKG-INFO                 |
|   | requires.txt             |
|   | SOURCES.txt              |
|   | top_level.txt            |
| ▼ | tests                    |
|   | test_geojson_nodes.py    |
|   | wymagania.pdf            |
|   | zabka_locs.geojson       |

### 3.5 Architektura systemu

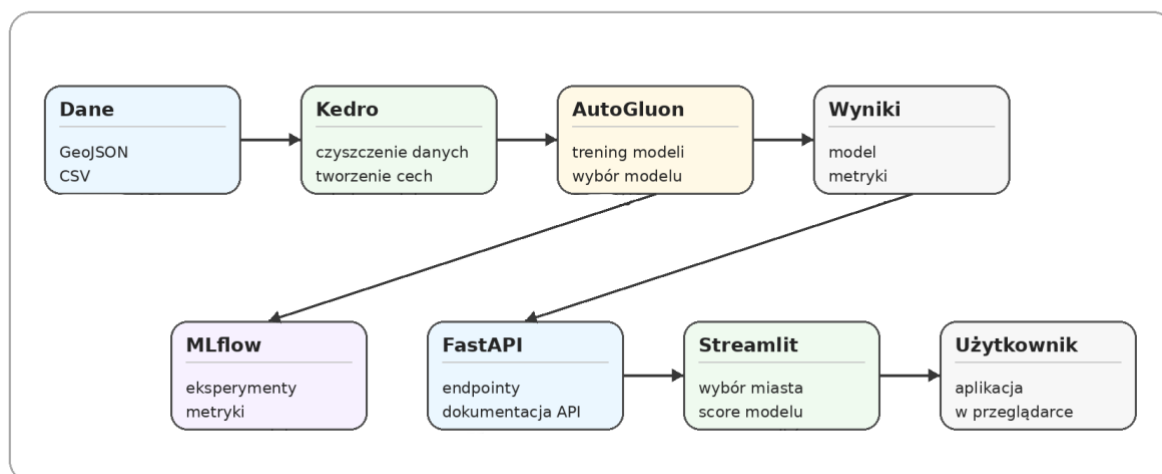
System składa się z kilku warstw: warstwy danych, warstwy przetwarzania, warstwy modelu oraz warstwy aplikacyjnej. Dane wejściowe pochodzą z pliku GeoJSON oraz plików CSV. Następnie są przetwarzane w pipeline Kedro, gdzie wykonywane jest czyszczenie danych, przygotowanie cech oraz zapis danych wejściowych dla modelu.

Model uczenia maszynowego jest trenowany na przygotowanych danych i zapisuje wyniki w katalogach projektu. Wyniki modelu są następnie udostępniane przez API FastAPI oraz prezentowane użytkownikowi w aplikacji Streamlit. Dodatkowo projekt wykorzystuje monitoring driftu danych oraz może zostać uruchomiony w kontenerach Docker.

Główne elementy systemu:

- dane wejściowe: GeoJSON oraz pliki CSV,
- Kedro: przetwarzanie danych i pipeline,
- AutoGluon: model docelowy,
- MLflow: śledzenie eksperymentów,
- FastAPI: udostępnienie wyników przez API,
- Streamlit: interfejs użytkownika,
- Docker: uruchomienie aplikacji w kontenerach.

#### Diagram architektury systemu





## 4. Sposób uruchomienia projektu

Projekt można uruchomić na kilka sposobów. W repozytorium znajduje się instrukcja w pliku README.md, która opisuje uruchomienie lokalne, uruchomienie API, uruchomienie interfejsu Streamlit oraz uruchomienie całego projektu za pomocą Dockera.

### 4.1 Instalacja lokalna

Pierwszym sposobem jest uruchomienie projektu lokalnie w środowisku Python. W tym celu należy utworzyć środowisko wirtualne, aktywować je, zainstalować wymagane biblioteki oraz zainstalować projekt w trybie edytowalnym.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -e .
```

Po wykonaniu tych komend projekt jest przygotowany do uruchomienia pipeline, API oraz interfejsu użytkownika.

### 4.2 Uruchomienie pipeline Kedro

Pipeline projektu odpowiada za przetwarzanie danych, przygotowanie cech, trenowanie modelu oraz wygenerowanie wyników. Uruchamia się go komendą:

```
kedro run --pipeline site_selection
```

Po zakończeniu działania pipeline generowane są m.in. dane wynikowe oraz ranking najlepszych kandydackich lokalizacji.

### 4.3 Uruchomienie API i mapy

Po przygotowaniu danych oraz modelu można uruchomić API aplikacji. API zostało przygotowane z użyciem FastAPI i pozwala na wykonywanie predykcji oraz dostęp do wyników przez przeglądarkę.

```
uvicorn zabki_prediction.api.main:app --reload
```

Po uruchomieniu dostępne są następujące adresy:

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/map
http://127.0.0.1:8000/monitoring/drift
```

Pierwszy adres prowadzi do dokumentacji API, drugi wyświetla mapę kandydackich lokalizacji, a trzeci służy do sprawdzenia monitoringu driftu.

## 4.4 Uruchomienie interfejsu Streamlit

Projekt posiada również prosty interfejs graficzny przygotowany w Streamlit. Interfejs pozwala wybrać miasto, sprawdzić wynik modelu oraz zobaczyć ranking kandydackich lokalizacji.

```
streamlit run src/zabki_prediction/ui/streamlit_app.py
```

Po uruchomieniu aplikacja Streamlit otwiera się w przeglądarce. Wynik prezentowany w aplikacji jest scorem modelu, czyli oceną atrakcyjności lokalizacji, a nie gwarancją rzeczywistego otwarcia sklepu.

## 4.5 Uruchomienie projektu przez Docker

**Najprostszą metodą uruchomienia całego projektu jest użycie Dockera.** W tym wariancie aplikacja uruchamiana jest w kontenerach na podstawie pliku docker-compose.yml.

```
docker compose up --build
```

Po uruchomieniu dostępne są:

```
Streamlit GUI: http://127.0.0.1:8501
FastAPI docs: http://127.0.0.1:8000/docs
FastAPI mapa: http://127.0.0.1:8000/map
```

Aby zatrzymać kontenery, należy użyć komendy:

```
docker compose down
```

W trakcie budowania obrazu Docker uruchamiany jest pipeline Kedro, dlatego model oraz rankingi kandydackich lokalizacji są tworzone wewnątrz kontenera.

## 5. Model uczenia maszynowego

W projekcie zastosowano model uczenia maszynowego do oceny atrakcyjności kandydackich lokalizacji pod nowe sklepy Żabka. Problem został potraktowany jako klasyfikacja binarna, w której istniejące lokalizacje sklepów oznaczono jako przykłady pozytywne, a wygenerowane punkty kandydackie jako przykłady negatywne.

### 5.1 Model bazowy

Jako model bazowy wykorzystano klasyfikator Random Forest. Model bazowy pozwolił sprawdzić, czy przygotowane cechy przestrzenne niosą informację przydatną do rozróżniania istniejących lokalizacji sklepów od punktów kandydackich.

Model bazowy został wytrenowany na danych przygotowanych przez pipeline. Dane zostały podzielone na zbiór treningowy i walidacyjny w proporcji 80/20.

Przykładowe wyniki modelu bazowego:

| Metryka           | Wynik  |
|-------------------|--------|
| ROC AUC           | 0.9899 |
| Average Precision | 0.9837 |
| Accuracy          | 0.9596 |
| F1-score          | 0.9387 |

Wysokie wyniki oznaczają, że przygotowane cechy dobrze rozróżniają istniejące lokalizacje sklepów od wygenerowanych punktów kandydackich. Należy jednak pamiętać, że problem został zbudowany na podstawie obecnej sieci sklepów i punktów wygenerowanych automatycznie, dlatego wynik nie oznacza stuprocentowej pewności biznesowej.

### 5.2 Model docelowy

Modelem docelowym w projekcie jest `AutoGluon TabularPredictor`. Jest to rozwiązanie AutoML, które automatycznie trenuje i porównuje różne modele tabularne, a następnie wybiera najlepszy wariant na podstawie zadanej metryki.

W projekcie jako metrykę ewaluacyjną zastosowano `roc_auc`, ponieważ model zwraca score atrakcyjności lokalizacji. Wynik modelu jest następnie używany do posortowania kandydackich punktów i przygotowania rankingu najlepszych lokalizacji.

Najważniejsze elementy modelowania:

- problem: klasyfikacja binarna,
- model bazowy: Random Forest,
- model docelowy: AutoGluon TabularPredictor,
- metryka główna: ROC AUC,
- wynik końcowy: ranking kandydackich lokalizacji,
- zapis wyników: katalog data/07\_model\_output.

### 5.3 Interpretacja wyniku modelu

Wynik zwracany przez model należy traktować jako score atrakcyjności lokalizacji. Im wyższy score, tym bardziej dana lokalizacja przypomina miejsca, w których obecnie znajdują się sklepy Żabka.

Nie jest to jednak gwarancja rzeczywistego otwarcia sklepu w danym miejscu. Model nie uwzględnia wszystkich czynników biznesowych, takich jak dostępność lokalu, koszt najmu, konkurencja bezpośrednia, umowy handlowe czy decyzje strategiczne firmy. Projekt należy więc traktować jako prototyp systemu wspierającego analizę lokalizacji.

### 5.4 Porównanie modeli AutoGluon

| Model             | ROC AUC walidacyjny | Czas treningu | Uwagi                                      |
|-------------------|---------------------|---------------|--------------------------------------------|
| RandomForest Gini | 0.9873              | 5.35 s        | Najlepszy pojedynczy model                 |
| RandomForestEntr  | 0.9868              | 2.43 s        | Wariant Random Forest z kryterium entropii |
| ExtraTreesGini    | 0.9820              | 0.93 s        | Model Extra Trees z kryterium gini         |
| ExtraTreesEntr    | 0.9827              | 0.99 s        | Model Extra Trees z kryterium entropii     |

| Model               | ROC AUC walidacyjny | Czas treningu | Uwagi                             |
|---------------------|---------------------|---------------|-----------------------------------|
| WeightedEnsemble_L2 | 0.9876              | 0.04 s        | Najlepszy model końcowy AutoGluon |

Najlepszy wynik walidacyjny uzyskał model WeightedEnsemble\_L2, czyli model zespołowy łączący kilka modeli bazowych. W jego skład weszły głównie modele RandomForestGini, RandomForestEntr oraz ExtraTreesEntr.

#### Wyniki końcowe modelu:

| Metryka                     | Wynik                    | Znaczenie                                                                                 |
|-----------------------------|--------------------------|-------------------------------------------------------------------------------------------|
| ROC AUC                     | 0.9906                   | Główna metryka modelu; ocenia zdolność rozróżniania lokalizacji pozytywnych i negatywnych |
| Average Precision           | 0.9845                   | Ocena jakości rankingu lokalizacji uznanych przez model za atrakcyjne                     |
| Zarejestrowana nazwa modelu | zabka-location-autogluon | Nazwa modelu zapisanego w projekcie                                                       |

Powyższe wyniki pochodzą z pliku `model_metrics.json` po odpaleniu pipeline kedro.

## 6. API i monitoring

Po wytrenowaniu modelu wyniki są udostępniane przez aplikację FastAPI. API pozwala korzystać z projektu bez konieczności bezpośredniego uruchamiania kodu pipeline. Użytkownik może sprawdzić dokumentację API w przeglądarce oraz korzystać z endpointów przygotowanych w projekcie.

Najważniejsze adresy aplikacji:

| Adres                                                                                       | Opis                               |
|---------------------------------------------------------------------------------------------|------------------------------------|
| <a href="http://127.0.0.1:8000/docs">http://127.0.0.1:8000/docs</a>                         | dokumentacja API FastAPI           |
| <a href="http://127.0.0.1:8000/map">http://127.0.0.1:8000/map</a>                           | mapa kandydackich lokalizacji      |
| <a href="http://127.0.0.1:8000/monitoring/drift">http://127.0.0.1:8000/monitoring/drift</a> | endpoint monitoringu driftu danych |
| <a href="http://127.0.0.1:8501">http://127.0.0.1:8501</a>                                   | interfejs użytkownika Streamlit    |

Projekt posiada również prosty monitoring danych. Monitoring pozwala porównać nowe dane wejściowe z profilem danych treningowych i sprawdzić, czy nie pojawiają się istotne różnice w rozkładach cech. Jest to ważny element podejścia MLOps, ponieważ w praktycznych systemach uczenia maszynowego jakość danych może zmieniać się w czasie.

Dodatkowo predykcje oraz wyniki projektu są zapisywane w katalogu `data/08_reporting`, który służy do przechowywania raportów, metryk oraz danych pomocniczych.

## 7. Podsumowanie

W ramach projektu przygotowano prototyp systemu wspierającego ocenę potencjalnych lokalizacji sklepów Żabka w Polsce. Projekt obejmuje cały podstawowy proces pracy z rozwiązaniem uczenia maszynowego: przygotowanie własnego datasetu z Overpass API, przetwarzanie danych w pipeline Kedro, trenowanie modelu AutoGluon, zapis wyników oraz udostępnienie aplikacji przez FastAPI i Streamlit.

Najważniejszym elementem projektu było dopasowanie aplikacji do własnych danych w formacie GeoJSON oraz przygotowanie cech przestrzennych opisujących analizowane lokalizacje. Model wykorzystuje między innymi informacje o położeniu punktu, odległości od istniejących sklepów, zagęszczeniu sklepów w okolicy oraz danych o liczbie mieszkańców.

Uzyskane wyniki pokazują, że przygotowane cechy pozwalają dobrze rozróżniać istniejące lokalizacje sklepów od wygenerowanych punktów kandydackich. Należy jednak pamiętać, że projekt ma charakter prototypowy. Wynik modelu powinien być traktowany jako wsparcie analizy lokalizacji, a nie jako jednoznaczna decyzja biznesowa o otwarciu sklepu.